

Project 3: Machine Learning for Predicting Trading Signals

Question: can the price features prepared in Project 2 predict rule-based Buy, Sell and Hold signals created by the MACD and RSI technical indicators?

Only very weakly. The rules produced an extremely imbalanced target (99.91% Hold). All three baseline models learned to predict Hold for every row, reaching 99.9% accuracy while finding zero Buy or Sell signals. Addressing the imbalance made the models find real signals (the improved Random Forest caught 43% of Buy and 52% of Sell signals on the test set), but at the cost of hundreds of thousands of false alarms. The features carry some information, far better than random guessing, but not enough to reproduce the rules reliably.

1. Data and Scope

The source is **historical_stock_prices.csv** (20,973,889 daily rows, 1970–2018). To stay consistent with Project 2, the analysis uses the same scope: **January 2010 to August 2018** (9,338,934 rows, 5,685 tickers).

Project 2's saved file (**stock_prepared.csv**) could not be reused directly because it was saved without the **ticker** and **date** columns. Project 3 needs both: MACD and RSI must be calculated separately for every stock, and the train/test split must follow time. The raw prices were therefore reloaded, the Project 2 cleaning decisions were re-applied, and the six Project 2 features were rebuilt with the same formulas.

Cleaning step	Result
Ticker hygiene (strip spaces, upper-case)	Applied (same as Project 2)
Missing dates/prices and duplicate ticker + date rows	None found
Contradictory price records (e.g. low above high, close outside the day's range)	1,348 rows removed
Final cleaned dataset	9,337,586 rows, 5,685 tickers, 2010-01-04 to 2018-08-24

2. Technical Indicators

Both indicators were coded by hand from their formulas in Python and Pandas, without a technical-analysis library and without the built-in **ewm()** function. A single manual Exponential Moving Average (EMA) function loops through the sorted data and restarts at the first row of every ticker, so one stock never influences another. It was verified to give identical results to the standard pandas calculation.

Exponential Moving Average

$EMA_t = \alpha \times value_t + (1 - \alpha) \times EMA_{t-1}$, where $\alpha = 2 / (\text{span} + 1)$. An EMA gives more weight to recent prices than a simple average does.

MACD (Moving Average Convergence Divergence)

MACD = $EMA_{12}(\text{close}) - EMA_{26}(\text{close})$, and the signal line = $EMA_9(\text{MACD})$. A **Buy** occurs on the day MACD crosses above its signal line (yesterday at or below, today above); a **Sell** occurs on the day it crosses below. Every other day is **Hold**.

RSI (Relative Strength Index, 14 days)

Daily price changes were split into gains and losses, each smoothed with an exponential average (Wilder's smoothing, $\alpha = 1/14$). $RS = \text{average gain} / \text{average loss}$, and $RSI = 100 - 100 / (1 + RS)$. RSI below 30 (oversold) is a **Buy**, above 70 (overbought) is a **Sell**, and anything between is **Hold**.

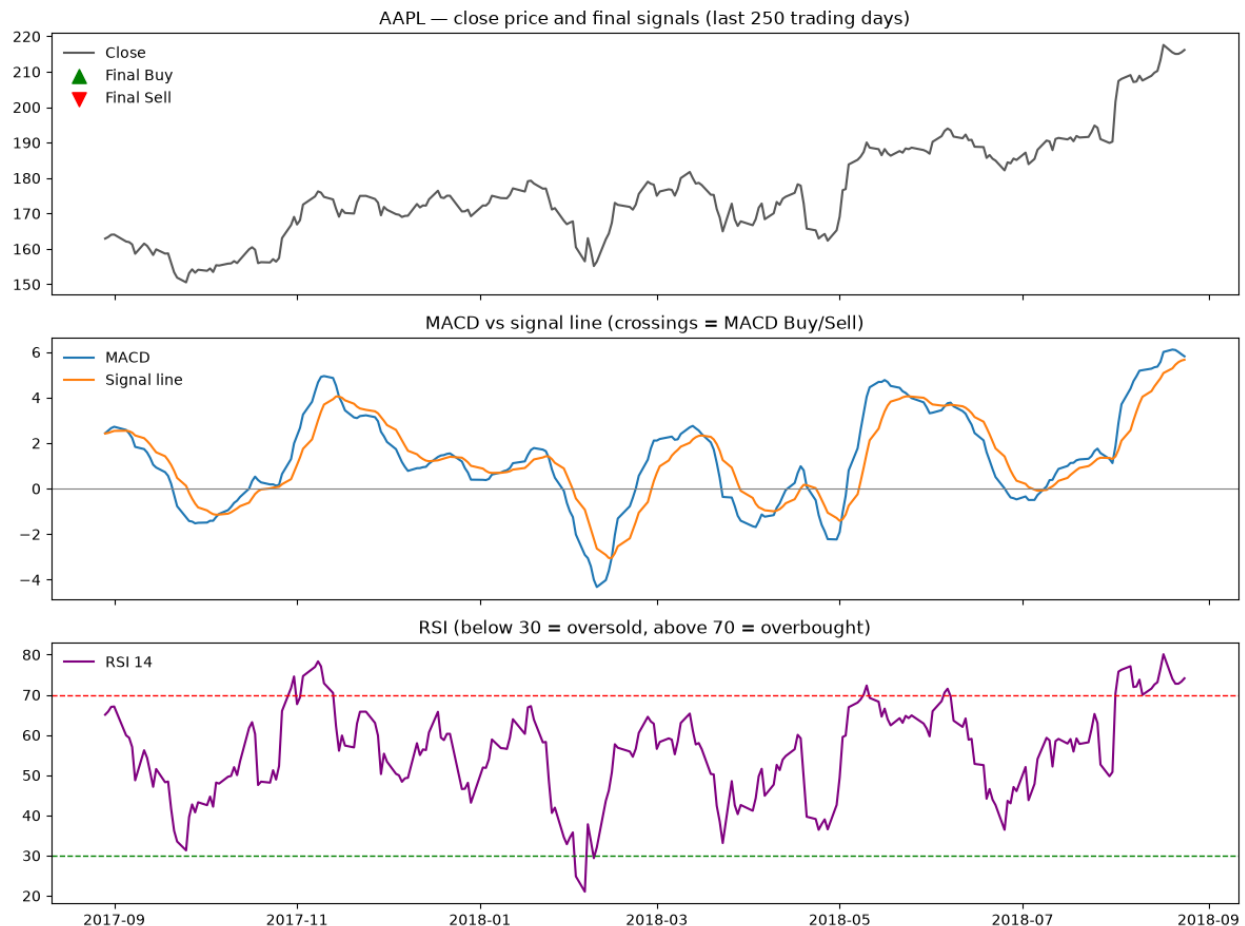


Figure 1. AAPL, last 250 trading days: close price, MACD vs signal line, and RSI. MACD crosses its signal line many times and RSI touches 30 and 70 several times, yet no day meets both conditions, so no final Buy or Sell appears.

Questions Answer the following:

1. How many Buy, Sell, and Hold signals were generated?

Q1. How many Buy, Sell and Hold signals were generated?

```
Buy :      3,170  (0.0355%)
Sell:     4,596  (0.0514%)
Hold:  8,933,470 (99.9131%)
Total rows: 8,941,236
```

2. Which signal occurs most frequently?

- Q2. Which signal occurs most frequently?
 Hold – 99.91% of all rows.
3. Why might Buy and Sell signals be much less common than Hold?
- Q3. Evidence for why Buy/Sell are rare:
 MACD crossed ABOVE its signal line on 368,587 days (4.12% of rows)
 RSI was below 30 on 348,626 days (3.90% of rows)
 Both on the SAME day (final Buy): 3,170
 Median RSI on MACD Buy-crossover days: 54.0 (a final Buy needs < 30)
 MACD crossed BELOW on 368,189 days; RSI above 70 on 523,153 days; both: 4,596
 Median RSI on MACD Sell-crossover days: 48.1 (a final Sell needs > 70)
4. What does this tell you about the distribution of your target variable?
- Q4. Distribution of the target:
 About 2,818 Hold rows for every Buy row and 1,944 for every Sell row.
 A model that always predicts Hold would already be 99.91% 'accurate'.

3. Trading Signal Creation

The two indicator signals were combined into one target, **trading_signal**: Buy only when both MACD and RSI say Buy on the same day, sell only when both say Sell, and Hold for any other combination. The first 35 rows of each ticker were removed as a warm-up period, because the 26-day EMA and 9-day signal line need time before their values are reliable. This left 8,941,236 rows.

Signal	MACD alone	RSI alone	Final (both agree)	Share of rows
Buy	368,587 (4.12%)	348,626 (3.90%)	3,170	0.0355%
Sell	368,189	523,153	4,596	0.0514%
Hold	—	—	8,933,470	99.9131%

Why Buy and Sell are so rare. Each indicator fires on about 4% of days, but they rarely fire on the same day. A MACD crossover is a one-day event, and it usually happens at a different moment than an RSI extreme. RSI drops below 30 *during* a strong fall, while MACD crosses upward only *after* the price has started to recover. By then RSI had usually climbed back up: the median RSI on MACD Buy-crossover days was **54.0**, far above the 30 needed for a final Buy. The same pattern holds for Sell signals (median RSI **48.1** on MACD Sell-crossover days, versus the 70 required).

```

              count  percent
trading_signal
Buy           3170    0.0355
Hold      8933470    99.9131
Sell         4596    0.0514

```

```

Buy:      3,170  (0.0355%)  majority-to-class ratio = 2,818 : 1
Hold: 8,933,470 (99.9131%)  majority-to-class ratio = 1 : 1
Sell:   4,596  (0.0514%)  majority-to-class ratio = 1,944 : 1

```

4. Class Imbalance

The target is extremely imbalanced: there are about **2,818 Hold rows for every Buy row** and **1,944 for every Sell row**. A model that always predicts Hold would already be 99.91% accurate while being useless for the signals that matter. The approach taken was:

First, detect the imbalance and report class counts and percentages. Second, keep the validation and test sets representative of real data (they were never resampled). Third, train baseline models without balancing to expose the "high accuracy trap", then address the imbalance with class weights and a more balanced training sample. Fourth, judge every model on per-class precision, recall, F1 and confusion matrices, not on accuracy alone.

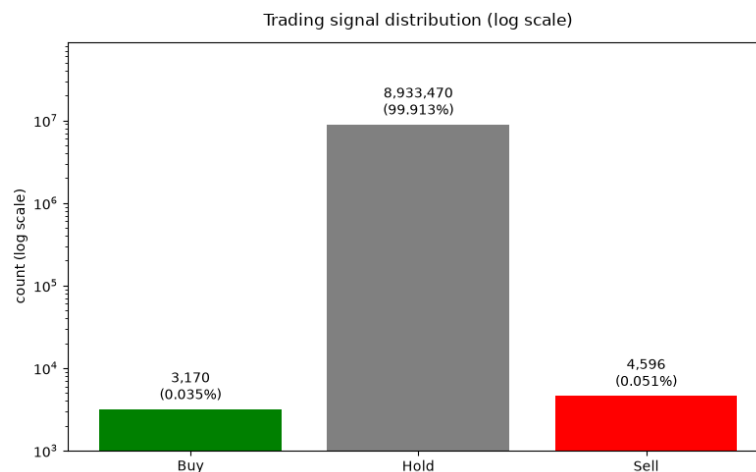


Figure 2. Distribution of the final trading signal (log scale; on a normal scale the Buy and Sell bars would be invisible).

5. Data Preparation for Machine Learning

Features and target

The six Project 2 features were used: **daily_return**, **close_lag_1**, **ma_5**, **ma_20**, **volatility_20** and **daily_range_pct**. MACD, RSI and their signals were deliberately excluded, and a code check confirms none of them is in the feature list. Because they were used to creating the target, giving them to the model would be **target leakage**: the model could copy the rule and score almost perfectly without learning anything. The labels were mapped as **Buy = 0**, **Hold = 1**, **Sell = 2**.

Chronological split

Stock data is ordered in time, so rows were not shuffled. The same cut-off dates as Project 2 were used, and the date ranges do not overlap.

Set	Dates	Rows	Buy %	Sell %	Used for
Train	2010-04-15 to 2014-12-31	4,430,337	0.0262%	0.0494%	Fitting the models
Validation	2015-01-02 to 2016-12-30	2,354,977	0.0488%	0.0453%	Comparing and improving models
Test	2017-01-03 to 2018-08-24	2,155,922	0.0400%	0.0621%	One final check on unseen data

```

Train      4,430,337 rows   2010-04-15 -> 2014-12-31
Validation 2,354,977 rows   2015-01-02 -> 2016-12-30
Test       2,155,922 rows   2017-01-03 -> 2018-08-24
Overlap train/val: False | Overlap val/test: False

```

```

train %   val %   test %

```

trading_signal

Buy	0.0262	0.0488	0.0400
Hold	99.9244	99.9059	99.8979
Sell	0.0494	0.0453	0.0621

Outliers and scaling

Project 2 found extreme values caused by reverse splits and data errors (daily returns above 500%). These distort Logistic Regression and SVM, so each feature was capped at the **0.5th and 99.5th percentiles of the training data**. For example, daily_return was limited to the range -9.7% to +11.3%. No rows were removed. The features were then standardized with a StandardScaler. Both the caps and the scaler were learned from the training data only and then applied to validation and test, so no future information leaked into training.

	lower (0.5%)	upper (99.5%)
daily_return	-0.0966	0.1125
close_lag_1	0.9700	708.6207
ma_5	0.9840	718.2359
ma_20	1.0000	727.9355
volatility_20	0.0027	0.1392
daily_range_pct	0.0000	0.1846

```
Train mean after scaling (≈0): [ 0.  0. -0.  0. -0.  0.]
```

```
Train std after scaling (≈1): [1. 1. 1. 1. 1. 1.]
```

6. Machine Learning Models

Three classifiers were trained and compared. Because the training set has 4.4 million rows, each model was trained on a **stratified sample** in which every class keeps its original share (random_state = 42). This keeps each sample representative while keeping training time reasonable.

Model	How it works	Training rows	Training time
Logistic Regression	Linear model estimating the probability of each class	1,000,000	1.6 s
Random Forest	100 decision trees that vote; learns non-linear patterns	299,999	14.9 s
SVM (linear kernel)	Finds the boundary separating classes with the widest margin	500,000	1.8 s

```

Logistic Regression rows: 1,000,000 classes: {'Buy': 262, 'Hold': 999244, 'Sell': 494}
Random Forest      rows: 299,999 classes: {'Buy': 78, 'Hold': 299773, 'Sell': 148}
SVM (linear)       rows: 500,000 classes: {'Buy': 131, 'Hold': 499622, 'Sell': 247}

```

```

Logistic Regression trained in 1.6s
Random Forest        trained in 14.9s
SVM (linear)         trained in 1.8s

```

A linear-kernel SVM (LinearSVC) was chosen over an RBF kernel. An RBF SVM's training time grows roughly with the square of the number of rows, and predicting 2 million validation rows would take many hours. The linear version uses the same maximum-margin idea and scales to this dataset.

7. Evaluation Results

Baseline models (validation set)

All three baseline models reached 99.91% accuracy, identical to the "always predict Hold" reference. When their predictions were counted, Logistic Regression and the SVM predicted Hold for every one of the 2,354,977 validation rows, and the Random Forest predicted a single Buy. Buy and Sell precision, recall and F1 were all 0. Macro recall was 0.333 because it averages Hold (1.0) with Buy (0) and Sell (0). High accuracy did not mean the models had learned anything useful.

Final comparison on the test set (2017–2018)

Every model was evaluated once on the untouched test set. The "always Hold" reference accuracy on test was 99.90%.

Model	Accuracy	Macro F1	Buy Prec.	Buy Recall	Sell Prec.	Sell Recall
Logistic Regression	0.9990	0.333	0.000	0.000	0.000	0.000
Random Forest	0.9990	0.333	0.000	0.000	0.000	0.000
SVM (linear)	0.9990	0.333	0.000	0.000	0.000	0.000
Random Forest (improved)	0.7540	0.290	0.004	0.430	0.002	0.520
Logistic Regression (tuned)	0.9990	0.333	0.000	0.000	0.000	0.000
Random Forest (tuned)	0.9990	0.333	0.000	0.000	0.000	0.000
SVM (linear) (tuned)	0.9959	0.340	0.010	0.064	0.004	0.004

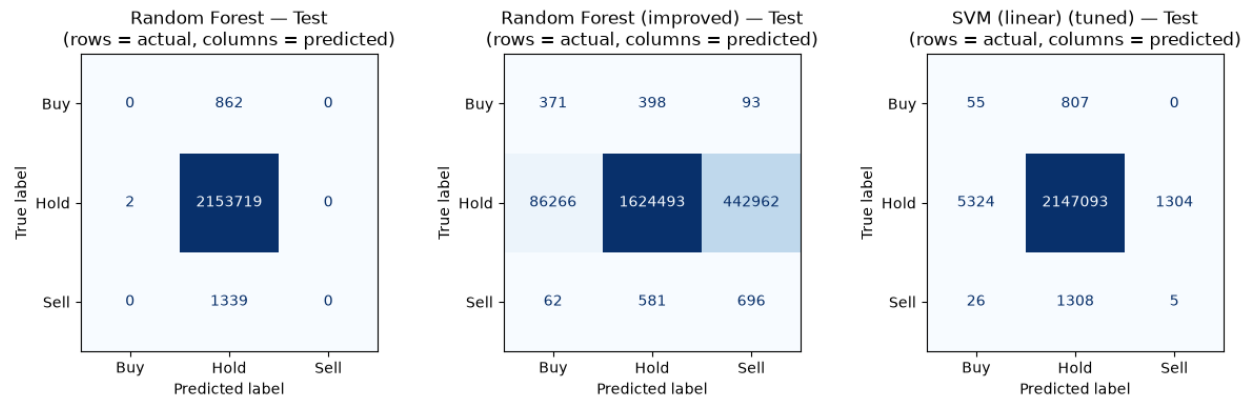


Figure 3. Test-set confusion matrices (rows = actual, columns = predicted). Left: baseline Random Forest, everything predicted as Hold. Centre: improved Random Forest, finds 371 of 862 Buys and 696 of 1,339 Sells but flags 86,266 and 442,962 ordinary days as Buy and Sell. Right: tuned SVM, more cautious, with fewer hits and far fewer false alarms.

8. Model Tuning and Optimization

Improving the Random Forest (Part 7)

The Random Forest was retrained with four changes, all judged on the validation set: a more balanced training sample (all 1,159 Buy and 2,190 Sell training rows plus 300,000 random Hold rows), **class_weight = "balanced_subsample"** so mistakes on rare classes cost more, **max_depth = 12** and **min_samples_leaf = 20** to limit memorising noise, and 200 trees instead of 100.

Validation metric	Before (baseline)	After (improved)	Change
Accuracy	0.9991	0.7925	-0.207
Buy recall	0.000	0.371	+0.371
Buy precision	0.000	0.005	+0.005
Sell recall	0.000	0.481	+0.481
Sell precision	0.000	0.001	+0.001
Macro F1	0.333	0.299	-0.035

Performance on the minority classes changed dramatically: the model went from finding no signals to finding about 4 in 10 Buys and half of the Sells. But precision stayed below 1%, so almost every predicted signal was a false alarm, and accuracy fell by 20 points. Macro F1 actually decreased, because the false alarms also hurt the Hold class. The same pattern held on the test set.

```
Part 7 – did performance on the minority classes change? (validation)
Buy: recall 0.000 -> 0.371, precision 0.000 -> 0.005, F1 0.000 -> 0.009
Sell: recall 0.000 -> 0.481, precision 0.000 -> 0.001, F1 0.000 -> 0.003
Accuracy 0.9991 -> 0.7925 | Macro F1 0.333 -> 0.299
```

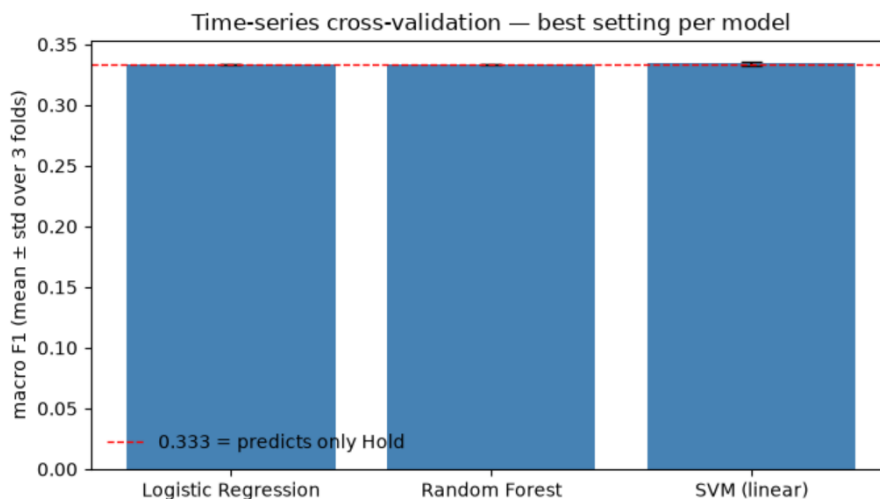
Hyperparameter search with time-series cross-validation

All three models were then tuned with **GridSearchCV** on a 300,000-row stratified training sample, using **TimeSeriesSplit** with 3 folds. Unlike normal K-fold, it always trains on earlier dates and validates on later

ones, so no future information is used. Models were scored on **macro F1** rather than accuracy, because macro F1 gives Buy, Hold and Sell equal weight.

Model	Settings searched	Best setting	CV macro F1 (mean $\pm$ std)
Logistic Regression	C: 0.01, 0.1, 1, 10 $\times$ class_weight: None / balanced	C = 0.01, no class weight	0.333 $\pm$ 0.000
Random Forest	max_depth: 8, 12 $\times$ min_samples_leaf: 20, 50 $\times$ class_weight	max_depth = 8, leaf = 20, no class weight	0.333 $\pm$ 0.000
SVM (linear)	C: 0.01, 0.1, 1 $\times$ class_weight: None / balanced	C = 0.01, balanced weights	0.334 $\pm$ 0.002

Cross-validation showed a consistent result across all three time periods: for Logistic Regression and Random Forest, turning on class weights *lowered* macro F1 (for example 0.186 to 0.248 for balanced Logistic Regression, versus 0.333 without weights), because the extra false alarms cost more than the few signals gained. The search therefore kept the "predict Hold" settings for those two models. Only the linear SVM with balanced weights beat the baseline, and on the test set it was the best model by macro F1 (0.340), with the highest Buy precision (1.0%) of all models.



9. Findings and Reflection

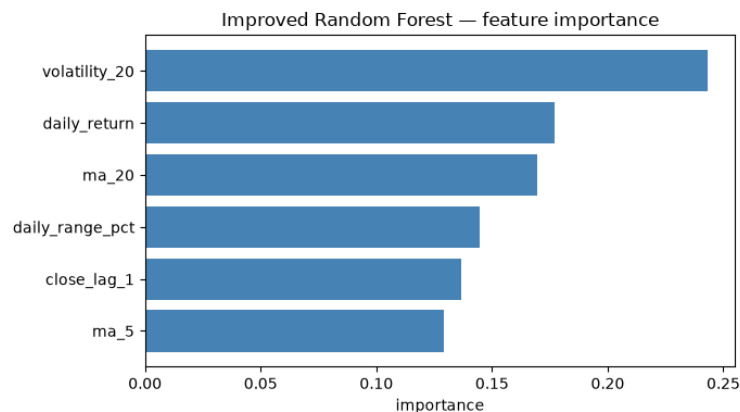


Figure 4. Feature importance from the improved Random Forest.

1. Which model performed differently across Buy, Hold and Sell?

The improved Random Forest. It was the only model with meaningful Buy recall (0.43) and Sell recall (0.52) on the test set, while its Hold recall fell to 0.75. The tuned SVM was the only other model to predict any signals, catching 6.4% of Buys. All other models behaved identically: perfect on Hold, zero on Buy and Sell.

Q1. Per-class recall on the test set:

	Buy Recall	Hold Recall	Sell Recall
Model			
Logistic Regression	0.0000	1.0000	0.0000
Random Forest	0.0000	1.0000	0.0000
SVM (linear)	0.0000	1.0000	0.0000
Random Forest (improved)	0.4304	0.7543	0.5198

Best model on the rare classes (mean Buy/Sell F1): Random Forest (improved) (0.006)

2. Did high accuracy correspond to good performance on Buy and Sell?

No. The highest-accuracy models (99.90%) found zero Buy or Sell signals; their accuracy exactly matched always guessing Hold. The model with the best Buy and Sell recall had the lowest accuracy (75.4%).

Q2. Did high accuracy correspond to good Buy/Sell performance?

Logistic Regression	accuracy 99.8979%		Buy recall 0.000		Sell recall 0.000
Random Forest	accuracy 99.8978%		Buy recall 0.000		Sell recall 0.000
SVM (linear)	accuracy 99.8979%		Buy recall 0.000		Sell recall 0.000
'Always Hold' accuracy for comparison: 99.8979%					

3. How did class imbalance affect the results?

With about 2,800 Hold rows per Buy row, the models learned that predicting Hold is almost always "right". The Random Forest's training sample contained only 78 Buy and 148 Sell rows out of 300,000, too few to learn from, so the minority classes were effectively ignored.

Q3/Q4. Random Forest before vs after addressing imbalance (test):

Accuracy	0.9990 -> 0.7540
Buy Recall	0.0000 -> 0.4304
Buy Precision	0.0000 -> 0.0043
Sell Recall	0.0000 -> 0.5198
Sell Precision	0.0000 -> 0.0016
Macro F1	0.3332 -> 0.2904

4. What happened when you addressed the imbalance?

The models started predicting the rare classes and recall rose sharply, but precision stayed below 1%. For every correctly found Buy, the improved Random Forest raised about 230 false Buy alarms. Addressing imbalance exposes a trade-off rather than solving the problem: catching more real signals always brought many more false ones.

5. Which features might be useful for predicting the signals?

volatility_20 (0.243) and **daily_return** (0.177) were the most important. This makes sense: RSI is built from the size of recent gains and losses, which these features also capture. The moving averages ranked lower because raw price levels (a \$5 stock versus a \$500 stock) are not comparable across tickers. Even so, the models did better than chance: the tuned SVM's Buy precision of 1.0% is about 25 times the 0.04% Buy base rate, so the features hold some real information.

Q5. Feature importance ranking (improved Random Forest):

1. volatility_20	0.243
2. daily_return	0.177
3. ma_20	0.169
4. daily_range_pct	0.145
5. close_lag_1	0.136
6. ma_5	0.129

6. What are the limitations of using MACD/RSI rules to create the target?

The settings (12/26/9 days, 30/70) are conventions, not proven values. Both indicators lag, reacting after price moves happen. The labels were never checked against what prices did next, so a Buy label does not mean the price rose. Requiring both indicators to agree on the same day made the target extremely rare. The models learn to copy a formula rather than the market, and data errors such as reverse splits can create false crossovers and extreme RSI values.

7. What additional features could be explored in a future project?

Scale-free ratios such as $\text{close} / \text{ma}_{20}$ and $\text{ma}_5 / \text{ma}_{20}$ (close to what MACD measures); returns over several days (ret_{5d} , ret_{21d}); volume features ($\log_{\text{vol}}$, vol_spike); the volatility ratio; position within a recent range (Bollinger position); sector and market-index returns; and news sentiment. Many of these were already built in Project 2. A more useful target would also be predicting *tomorrow's* signal rather than today's.

8. Would these model predictions guarantee profitable trading?

No. The models predict a rule-based label, not future returns, and the rules were never shown to be profitable. Buy and Sell precision was below 1.1% for every model, so almost all predicted signals would be false alarms. Real trading also involves transaction costs, slippage, taxes and risk, none of which were modelled, and patterns learned from 2010–2014 may not hold in later markets.

10. Conclusion and Limitations

This project built a complete machine-learning classification workflow: manual MACD and RSI calculation, rule-based signal creation, leakage-safe feature preparation, a chronological train/validation/test split, three models, class-imbalance handling, and hyperparameter tuning with time-series cross-validation.

The central finding is that the Project 2 features **cannot reliably predict** the MACD/RSI signals. The target is so rare (0.09% Buy or Sell) that accuracy is meaningless, and every attempt to find more signals produced far more false alarms. The best model (tuned linear SVM, macro F1 0.340) is only slightly better than predicting Hold every day (0.333), although its precision well above the base rate shows the features contain some real information.

Limitations of this analysis: models were trained on samples of the training data rather than all 4.4 million rows; a linear rather than RBF kernel was used for the SVM for computational reasons; hyperparameter grids were small; outliers were capped rather than corrected at the source; and the target itself inherits all the limitations of the MACD/RSI rules. The most promising next steps are scale-free ratio features, a less strict target (for example, either indicator firing), and a target that measures what actually happens to the price afterwards.